Hyperledger Fabric RBAC System

Blockchain Assignment Report

Project Guide:

Dr. Sourav Kanti Addya

Department of Computer Science and Engineering

Submitted by

Chinthakommadinne Vinay Kumar	(252IS010)
Perla Venkata Laxmi Subha Satya Anusha	(252IS025)
Ajay Shaji	(252IS002)

Department of Computer Science and Engineering

National Institute of Technology Karnataka, Surathkal

Mangalore - 575025, Karnataka, India

April 2026

Contents

1 Introduction to Hyperledger Fabric **2**
1.1 What is Hyperledger Fabric? . . . 2
1.2 How Hyperledger Fabric Works . . . 2
1.3 Core Components . . . 3
1.4 Why Hyperledger Fabric for Enterprise? . . . 3
1.5 Real-World Use Cases . . . 4

2 Problem Statement **4**
2.1 The Challenge . . . 4
2.2 Assignment Requirements . . . 4
2.3 Role Permissions Matrix . . . 5

3 How RBAC Works Using the CID Library **5**
3.1 The Client Identity Library . . . 5
3.2 Why This Approach is Secure . . . 5
3.3 How the Role Gets Into the Certificate . . . 6

4 Technology Stack **6**

5 Project Structure **6**

6 Implementation **7**
6.1 Chaincode Functions . . . 7
6.2 Core RBAC Chaincode . . . 7

7 Setup and Deployment **9**
7.1 Prerequisites . . . 9
7.2 Step 1 — Deploy Network and Chaincode . . . 9
7.3 Step 2 — Set Environment Variables . . . 11
7.4 Step 3 — Register Users with Role Attributes . . . 12
7.5 Step 4 — Enroll Users and Fix MSP . . . 13

8 Testing and Results **15**
8.1 Test 1 — Admin Creates an Asset . . . 15
8.2 Test 2 — Auditor Reads an Asset . . . 16
8.3 Test 3 — Auditor Tries to Create (Unauthorized) . . . 17
8.4 Test 4 — Manager Updates an Asset . . . 18
8.5 Test 6 — Frontend Demo . . . 19

9 Conclusion **20**

1 Introduction to Hyperledger Fabric

1.1 What is Hyperledger Fabric?

Hyperledger Fabric is an open-source, enterprise-grade permissioned blockchain framework hosted by the Linux Foundation under the Hyperledger umbrella project. Unlike public blockchains such as Bitcoin or Ethereum — where anyone can join the network anonymously — Fabric is designed for business use cases where participants are known, trusted, and authenticated using digital certificates.

Fabric was originally contributed by IBM and Digital Asset and has since grown into one of the most widely adopted enterprise blockchain platforms in the world. It provides a modular, pluggable architecture that allows organizations to tailor the network to their specific needs.

Key Distinction: Hyperledger Fabric is a **permissioned** blockchain. Every participant holds a cryptographic identity issued by a Certificate Authority. There is no anonymous access, no mining, and no cryptocurrency. This makes it ideal for enterprise applications requiring accountability and regulatory compliance.

1.2 How Hyperledger Fabric Works

Every transaction in Fabric goes through a strict three-phase lifecycle called **Execute-Order-Validate**:

1. **Execute (Endorse)** — The client application submits a transaction proposal to endorsing peers. Each peer simulates the chaincode and returns a signed response without writing to the ledger yet.

2. **Order** — The client collects endorsements and submits the transaction to the Ordering Service (Raft). The orderer sequences transactions into blocks and distributes them to all peers.

3. **Validate and Commit** — Every peer validates each transaction against the endorsement policy, checks for conflicts, and commits valid transactions to the ledger.

1.3 Core Components

Table 1: Hyperledger Fabric Core Components

Component	Description
Peer	Node that hosts the ledger and runs chaincode
Orderer	Orders transactions into blocks using Raft consensus
Fabric CA	Certificate Authority — issues X.509 certificates to all participants
Channel	Private subnet between organizations, each with its own ledger
Chaincode	Smart contract — business logic written in Go, Java, or Node.js
MSP	Membership Service Provider — manages digital identities
World State	Current state of the ledger stored as a key-value store
Ledger	World state (current data) plus blockchain (immutable history)

1.4 Why Hyperledger Fabric for Enterprise?

- **Permissioned Network** — All participants are identified using X.509 certificates. No anonymous transactions.
- **No Cryptocurrency** — No mining fees or gas costs, making it practical for business.
- **Private Data** — Sensitive data can be shared only between specific organizations.
- **High Throughput** — Thousands of transactions per second, suitable for production.
- **Pluggable Consensus** — The consensus mechanism can be swapped without redesign.
- **Channels** — Multiple isolated ledgers can coexist on the same physical network.

1.5 Real-World Use Cases

Table 2: Real-World Hyperledger Fabric Deployments

Industry	Use Case
Supply Chain	IBM Food Trust — tracks food from farm to store
Trade Finance	We.Trade — automates cross-border transactions between banks
Healthcare	Patient record sharing between hospitals with privacy controls
Government	Tamper-proof land registry and property ownership records
Finance	Interbank settlement and KYC data sharing
Insurance	Automated claims processing using smart contracts

2 Problem Statement

2.1 The Challenge

In any real-world enterprise blockchain network, not every participant should have equal access to every operation. Without proper access control:

- An auditor could accidentally or maliciously modify records they should only read.
- A manager could create unauthorized assets outside their scope of authority.
- There is no enforceable separation of duties, violating basic security principles.

The challenge is to implement certificate-based RBAC directly at the chaincode level — meaning access decisions are enforced by the smart contract on the blockchain itself, not just at the application layer.

Even if someone bypasses the frontend application and invokes the chaincode directly via CLI, the role check still applies — because it runs inside the peer before any ledger operation.

2.2 Assignment Requirements

Assignment Brief: Implement role-based access control in chaincode using the Client Identity Library.

Roles: Admin, Manager, Auditor

Requirement: Unauthorized operations must be rejected with clear error messages.

Marks: 17 total — Implementation (9), Documentation (4), Presentation (4)

2.3 Role Permissions Matrix

Table 3: RBAC Permission Matrix

Operation	Admin	Manager	Auditor
Create Asset	Yes	No	No
Update Asset	No	Yes	No
Read Asset	Yes	Yes	Yes
Delete Asset	Yes	No	No
Get All Assets	Yes	Yes	Yes

3 How RBAC Works Using the CID Library

3.1 The Client Identity Library

The solution uses the Hyperledger Fabric Client Identity (CID) Library, part of the official `fabric-chaincode-go` package. When a user is enrolled with the Fabric CA, a custom `role` attribute is embedded directly into their X.509 certificate. The chaincode reads this attribute on every function call.

Listing 1: Reading role from the caller's certificate

```
role, found, err := cid.GetAttributeValue(ctx.GetStub(), "role"
   )
if err != nil {
    return "", fmt.Errorf("error reading role attribute: %v",
       err)
}
if !found {
    return "", fmt.Errorf("role attribute not found in
       certificate")
}
```

3.2 Why This Approach is Secure

- The `role` attribute is embedded in the X.509 certificate, which is cryptographically signed by the Fabric CA and cannot be forged.
- The role check happens inside the peer before any ledger read or write.
- Even direct CLI invocations are subject to the same role enforcement.

- The ledger permanently records the identity of who performed each operation.

3.3 How the Role Gets Into the Certificate

The `:ecert` suffix during registration ensures the attribute is embedded in the enrollment certificate:

Listing 2: Registering a user with the role attribute in Fabric CA

```
fabric-ca-client register --caname ca-org1 \
  --id.name adminUser --id.secret adminpw \
  --id.type client \
  --id.attrs "role=Admin:ecert" \
  --tls.certfiles ./organizations/fabric-ca/org1/ca-cert.pem
```

4 Technology Stack

Table 4: Technologies and Tools Used

Technology	Version	Purpose
Hyperledger Fabric	v2.5	Core permissioned blockchain platform
Go (Golang)	v1.21	Language used to write the chaincode
fabric-chaincode-go	latest	Official Fabric Go SDK and CID library
fabric-contract-api-go	v1.2.2	High-level contract API for chaincode
Fabric CA	v1.5	Issues X.509 certificates with role attributes
Docker and Compose	v24+	Containerizes all Fabric network components
Node.js and Express.js	v18+	Backend REST API server
fabric-gateway (Node.js)	v1.4	Node.js SDK to submit transactions to Fabric
HTML / CSS / JavaScript	—	Frontend demo user interface
Ubuntu	22.04 LTS	Development and deployment environment

5 Project Structure

Listing 3: Project directory layout

```
~/fabric-samples/
|-- chaincode/
```

```
|   `-- rbac/
|       |-- smartcontract.go    <- Main RBAC chaincode logic
|       |-- go.mod              <- Go module definition
|       |-- go.sum              <- Dependency checksums
|       `-- vendor/             <- Vendored Go dependencies
|
`-- test-network/
    |-- network.sh              <- Start/stop network
    `-- organizations/          <- Crypto material and MSPs

~/rbac-project/
|-- README.md
|-- frontend/
|   `-- index.html              <- Demo UI
|-- backend/
|   |-- server.js               <- Express REST API
|   `-- package.json
`-- scripts/
    |-- deploy.sh               <- Deployment script
    `-- test-rbac.sh            <- CLI test suite
```

6 Implementation

6.1 Chaincode Functions

Table 5: Chaincode Functions and Access Control

Function	Access	Description
`CreateAsset(id, name, value)`	Admin only	Creates a new asset. Fails if ID already exists.
`UpdateAsset(id, name, value)`	Manager only	Updates an existing asset. Preserves original owner.
`ReadAsset(id)`	All roles	Returns a single asset by ID.
`GetAllAssets()`	All roles	Returns all assets on the ledger.
`DeleteAsset(id)`	Admin only	Permanently removes an asset.

6.2 Core RBAC Chaincode

Listing 4: getRole() helper function

```go
func getRole(ctx contractapi.TransactionContextInterface) (
   string, error) {
    role, found, err := cid.GetAttributeValue(ctx.GetStub(), "
       role")
    if err != nil {
        return "", fmt.Errorf("error reading role attribute: %v
           ", err)
    }
    if !found {
        return "", fmt.Errorf("role attribute not found in
           certificate")
    }
    return role, nil
}
```

Listing 5: CreateAsset — Admin only

```go
func (s *SmartContract) CreateAsset(
    ctx contractapi.TransactionContextInterface,
    id, name, value string) error {

    role, err := getRole(ctx)
    if err != nil {
        return fmt.Errorf("Access denied: %v", err)
    }
    if role != "Admin" {
        return fmt.Errorf("Access denied: only Admin can create
            assets"+
            " (your role: %s)", role)
    }

    existing, _ := ctx.GetStub().GetState(id)
    if existing != nil {
        return fmt.Errorf("asset %s already exists", id)
    }
    asset := Asset{ID: id, Name: name, Value: value}
    data, _ := json.Marshal(asset)
    return ctx.GetStub().PutState(id, data)
}
```

Listing 6: UpdateAsset — Manager only

```go
func (s *SmartContract) UpdateAsset(
    ctx contractapi.TransactionContextInterface,
    id, name, value string) error {

    role, err := getRole(ctx)
    if err != nil {
        return fmt.Errorf("Access denied: %v", err)
    }
    if role != "Manager" {
        return fmt.Errorf("Access denied: only Manager can
           update assets"+
            " (your role: %s)", role)
    }
    data, err := ctx.GetStub().GetState(id)
    if err != nil || data == nil {
        return fmt.Errorf("asset %s not found", id)
    }
    updated := Asset{ID: id, Name: name, Value: value}
    newData, _ := json.Marshal(updated)
    return ctx.GetStub().PutState(id, newData)
}
```

7 Setup and Deployment

7.1 Prerequisites

Listing 7: Verify all tools are installed

```
docker --version          # Need v24+
docker compose version    # Need v2+
go version                # Need v1.21+
node --version            # Need v18+
peer version              # Fabric binaries must be in PATH
fabric-ca-client version
```

7.2 Step 1 — Deploy Network and Chaincode

Listing 8: Deploy the Fabric network and chaincode

```
cd ~/fabric-samples/test-network
bash ~/rbac-project/scripts/deploy.sh
```

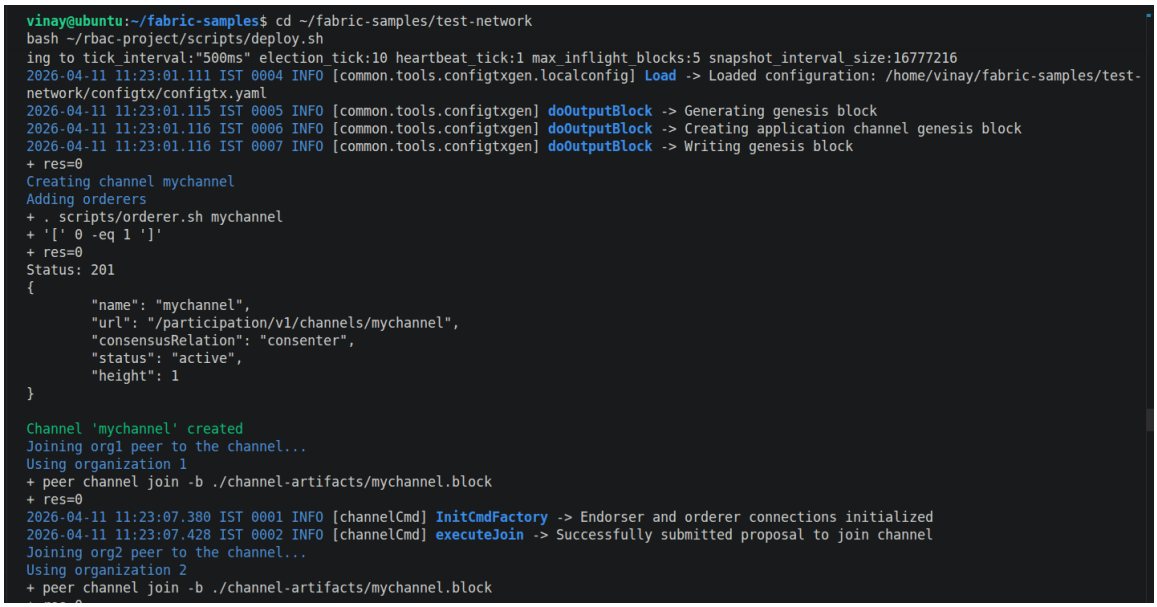
This script tears down any existing network, starts it fresh with CA enabled, creates `mychannel`, and deploys the `rbac` chaincode. You should see *"Chaincode deployed successfully!"* at the end.

```
vinay@ubuntu:~/fabric-samples$ cd ~/fabric-samples/test-network
bash ~/rbac-project/scripts/deploy.sh
▶ Step 2: Starting Fabric network with CA...
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto fro
m 'Certificate Authorities'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
[+] up 4/4
 ✔ Network fabric_test  Created                                                                                                    0.1s
 ✔ Container ca_org2    Started                                                                                                    0.8s
 ✔ Container ca_org1    Started                                                                                                    1.0s
 ✔ Container ca_orderer Started                                                                                                    0.9s
+ fabric-ca-client getcainfo -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/vinay/fabric-samples/test-netwo
rk/organizations/fabric-ca/org1/ca-cert.pem
2026/04/11 11:22:53 [INFO] Configuration file location: /home/vinay/fabric-samples/test-network/organizations/peerOrganizations/org1.exampl
e.com/fabric-ca-client-config.yaml
2026/04/11 11:22:53 [INFO] TLS Enabled
2026/04/11 11:22:53 [INFO] Stored root CA certificate at /home/vinay/fabric-samples/test-network/organizations/peerOrganizations/org1.examp
le.com/msp/cacerts/localhost-7054-ca-org1.pem
2026/04/11 11:22:53 [INFO] Stored Issuer public key at /home/vinay/fabric-samples/test-network/organizations/peerOrganizations/org1.example
.com/msp/IssuerPublicKey
2026/04/11 11:22:53 [INFO] Stored Issuer revocation public key at /home/vinay/fabric-samples/test-network/organizations/peerOrganizations/o
rg1.example.com/msp/IssuerRevocationPublicKey
+ res=0
Creating Org1 Identities
Enrolling the CA admin
+ fabric-ca-client enroll -u https://admin:adminpw@localhost:7054 --caname ca-org1 --tls.certfiles /home/vinay/fabric-samples/test-network/
```

Figure 1: Network Started

```
vinay@ubuntu:~/fabric-samples$ cd ~/fabric-samples/test-network
bash ~/rbac-project/scripts/deploy.sh
Generating CCP files for Org1 and Org2
[+] up 3/3
 ✔ Volume compose_peer0.org2.example.com Created                                                                                   0.0s
[+] up 6/6compose_orderer.example.com    Created                                                                                   0.0s
 ✔ Volume compose_peer0.org2.example.com Created                                                                                   0.0s
 ✔ Volume compose_orderer.example.com    Created                                                                                   0.0s
 ✔ Volume compose_peer0.org1.example.com Created                                                                                   0.0s
 ✔ Container peer0.org2.example.com      Started                                                                                   1.3s
 ✔ Container orderer.example.com         Started                                                                                   1.0s
 ✔ Container peer0.org1.example.com      Started                                                                                   1.2s
CONTAINER ID   IMAGE                              COMMAND                  CREATED          STATUS                  PORTS
                                                                                                                   NAMES
7a3979019fd0   hyperledger/fabric-peer:latest     "peer node start"        1 second ago     Up Less than a second   0.0.0.0:7051->7051/tcp
, [::]:7051->7051/tcp, 0.0.0.0:9444->9444/tcp, [::]:9444->9444/tcp                                                 peer0.org1.example.com
3f97d95d30ab   hyperledger/fabric-peer:latest     "peer node start"        1 second ago     Up Less than a second   0.0.0.0:9051->9051/tcp
, [::]:9051->9051/tcp, 7051/tcp, 0.0.0.0:9445->9445/tcp, [::]:9445->9445/tcp                                       peer0.org2.example.com
fd5a74fc9975   hyperledger/fabric-orderer:latest  "orderer"                1 second ago     Up Less than a second   0.0.0.0:7050->7050/tcp
, [::]:7050->7050/tcp, 0.0.0.0:7053->7053/tcp, [::]:7053->7053/tcp, 0.0.0.0:9443->9443/tcp, [::]:9443->9443/tcp   orderer.example.com
c5b87bce8584   hyperledger/fabric-ca:latest       "sh -c 'fabric-ca-se…"   10 seconds ago   Up 10 seconds           0.0.0.0:7054->7054/tcp
, [::]:7054->7054/tcp, 0.0.0.0:17054->17054/tcp, [::]:17054->17054/tcp                                             ca_org1
0c0e17745a8e   hyperledger/fabric-ca:latest       "sh -c 'fabric-ca-se…"   10 seconds ago   Up 10 seconds           0.0.0.0:8054->8054/tcp
, [::]:8054->8054/tcp, 7054/tcp, 0.0.0.0:18054->18054/tcp, [::]:18054->18054/tcp                                   ca_org2
0ffd2c55dbae   hyperledger/fabric-ca:latest       "sh -c 'fabric-ca-se…"   10 seconds ago   Up 10 seconds           0.0.0.0:9054->9054/tcp
, [::]:9054->9054/tcp, 7054/tcp, 0.0.0.0:19054->19054/tcp, [::]:19054->19054/tcp                                   ca_orderer
a401c8c9f5a8   fb2f304515cd                       "/bin/sh -c '#(nop) …"   37 hours ago     Created
                                                                                                                   mystifying_mestorf
05bb72cb1b89   6b0cef7f5d0f                       "/bin/sh -c '#(nop) …"   37 hours ago     Created
                                                                                                                   nervous_murdock
c6fc38839fe8   177c3da31f86                       "/bin/sh -c '#(nop) …"   37 hours ago     Created
                                                                                                                   hopeful_engelbart
ba0cefc7cd40   a9d96233d0cb                       "/bin/sh -c '#(nop) …"   37 hours ago     Created
```

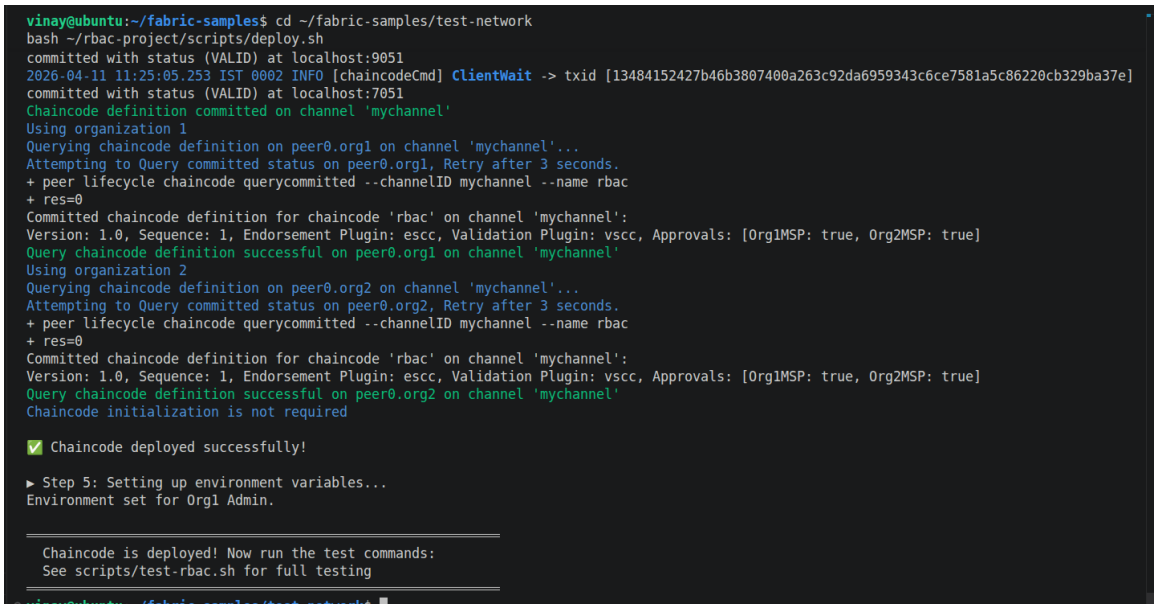
Figure 2



```
vinay@ubuntu:~/fabric-samples$ cd ~/fabric-samples/test-network
bash ~/rbac-project/scripts/deploy.sh
ing to tick_interval:"500ms" election_tick:10 heartbeat_tick:1 max_inflight_blocks:5 snapshot_interval_size:16777216
2026-04-11 11:23:01.111 IST 0004 INFO [common.tools.configtxgen.localconfig] Load -> Loaded configuration: /home/vinay/fabric-samples/test-
network/configtx/configtx.yaml
2026-04-11 11:23:01.115 IST 0005 INFO [common.tools.configtxgen] doOutputBlock -> Generating genesis block
2026-04-11 11:23:01.116 IST 0006 INFO [common.tools.configtxgen] doOutputBlock -> Creating application channel genesis block
2026-04-11 11:23:01.116 IST 0007 INFO [common.tools.configtxgen] doOutputBlock -> Writing genesis block
+ res=0
Creating channel mychannel
Adding orderers
+ . scripts/orderer.sh mychannel
+ '[' 0 -eq 1 ']'
+ res=0
Status: 201
{
        "name": "mychannel",
        "url": "/participation/v1/channels/mychannel",
        "consensusRelation": "consenter",
        "status": "active",
        "height": 1
}

Channel 'mychannel' created
Joining org1 peer to the channel...
Using organization 1
+ peer channel join -b ./channel-artifacts/mychannel.block
+ res=0
2026-04-11 11:23:07.380 IST 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-11 11:23:07.428 IST 0002 INFO [channelCmd] executeJoin -> Successfully submitted proposal to join channel
Joining org2 peer to the channel...
Using organization 2
+ peer channel join -b ./channel-artifacts/mychannel.block
```

Figure 3: Channel created



```
vinay@ubuntu:~/fabric-samples$ cd ~/fabric-samples/test-network
bash ~/rbac-project/scripts/deploy.sh
committed with status (VALID) at localhost:9051
2026-04-11 11:25:05.253 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [13484152427b46b3807400a263c92da6959343c6ce7581a5c86220cb329ba37e]
committed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name rbac
+ res=0
Committed chaincode definition for chaincode 'rbac' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name rbac
+ res=0
Committed chaincode definition for chaincode 'rbac' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

✅ Chaincode deployed successfully!

▶ Step 5: Setting up environment variables...
Environment set for Org1 Admin.

═══════════════════════════════════════════════
  Chaincode is deployed! Now run the test commands:
  See scripts/test-rbac.sh for full testing
═══════════════════════════════════════════════
```

Figure 4: Chaincode Deployed

7.3 Step 2 — Set Environment Variables

Run these in every new terminal session before using the peer CLI:

Listing 9: Environment variables for Org1

```
cd ~/fabric-samples/test-network

export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/
```

```
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/
   peerOrganizations/\
  org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/
   peerOrganizations/\
  org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

export ORDERER_CA=${PWD}/organizations/ordererOrganizations/
   example.com/\
  orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com
     -cert.pem
```

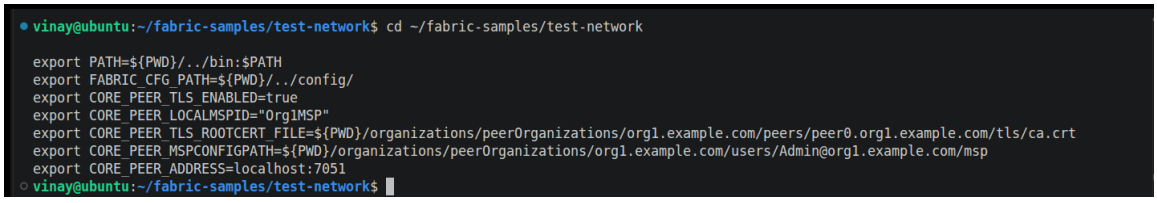


Figure 5

7.4 Step 3 — Register Users with Role Attributes

Listing 10: Register all three users with the Fabric CA

```
export FABRIC_CA_CLIENT_HOME=${PWD}/organizations/
   peerOrganizations/\
  org1.example.com/

# Register Admin
fabric-ca-client register --caname ca-org1 \
  --id.name adminUser --id.secret adminpw --id.type client \
  --id.attrs "role=Admin:ecert" \
  --tls.certfiles ${PWD}/organizations/fabric-ca/org1/ca-cert.
     pem

# Register Manager
fabric-ca-client register --caname ca-org1 \
  --id.name managerUser --id.secret managerpw --id.type client
     \
  --id.attrs "role=Manager:ecert" \
```

```
  --tls.certfiles ${PWD}/organizations/fabric-ca/org1/ca-cert.
      pem

# Register Auditor
fabric-ca-client register --caname ca-org1 \
  --id.name auditorUser --id.secret auditorpw --id.type client
      \
  --id.attrs "role=Auditor:ecert" \
  --tls.certfiles ${PWD}/organizations/fabric-ca/org1/ca-cert.
      pem
```

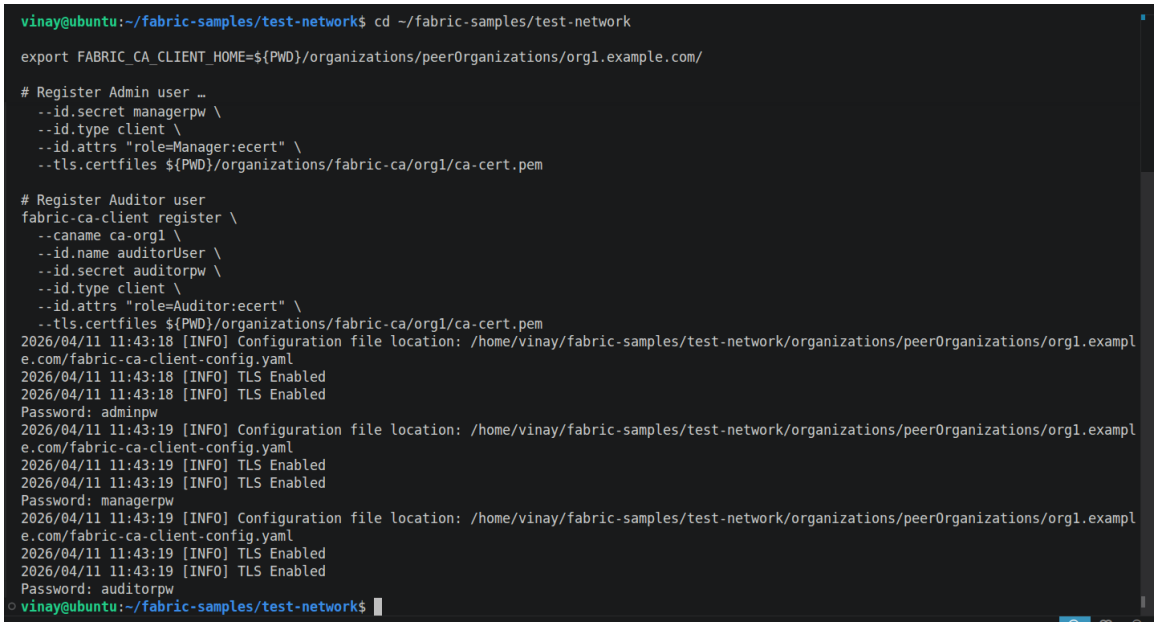


Figure 6

7.5 Step 4 — Enroll Users and Fix MSP

Why admincerts is required: After enrollment, you must manually create the `admincerts/` folder. The Fabric peer CLI requires this folder — it tells the peer that this certificate is authorized to act on behalf of the organization. Without it, the peer rejects the identity even if the certificate itself is valid.

Listing 11: Enroll AdminUser and fix MSP

```
fabric-ca-client enroll \
  -u https://adminUser:adminpw@localhost:7054 --caname ca-org1
      \
  -M ${PWD}/organizations/peerOrganizations/org1.example.com/\
    users/AdminUser@org1.example.com/msp \
  --enrollment.attrs "role" \
```

```
  --tls.certfiles ${PWD}/organizations/fabric-ca/org1/ca-cert.
      pem

mkdir -p organizations/peerOrganizations/org1.example.com/\
  users/AdminUser@org1.example.com/msp/admincerts
cp organizations/peerOrganizations/org1.example.com/\
  users/AdminUser@org1.example.com/msp/signcerts/cert.pem \
  organizations/peerOrganizations/org1.example.com/\
  users/AdminUser@org1.example.com/msp/admincerts/
```

Listing 12: Enroll ManagerUser and fix MSP

```
fabric-ca-client enroll \
  -u https://managerUser:managerpw@localhost:7054 --caname ca-
      org1 \
  -M ${PWD}/organizations/peerOrganizations/org1.example.com/\
    users/ManagerUser@org1.example.com/msp \
  --enrollment.attrs "role" \
  --tls.certfiles ${PWD}/organizations/fabric-ca/org1/ca-cert.
      pem

mkdir -p organizations/peerOrganizations/org1.example.com/\
  users/ManagerUser@org1.example.com/msp/admincerts
cp organizations/peerOrganizations/org1.example.com/\
  users/ManagerUser@org1.example.com/msp/signcerts/cert.pem \
  organizations/peerOrganizations/org1.example.com/\
  users/ManagerUser@org1.example.com/msp/admincerts/
```

Listing 13: Enroll AuditorUser and fix MSP

```
fabric-ca-client enroll \
  -u https://auditorUser:auditorpw@localhost:7054 --caname ca-
      org1 \
  -M ${PWD}/organizations/peerOrganizations/org1.example.com/\
    users/AuditorUser@org1.example.com/msp \
  --enrollment.attrs "role" \
  --tls.certfiles ${PWD}/organizations/fabric-ca/org1/ca-cert.
      pem

mkdir -p organizations/peerOrganizations/org1.example.com/\
  users/AuditorUser@org1.example.com/msp/admincerts
cp organizations/peerOrganizations/org1.example.com/\
```

```
users/AuditorUser@org1.example.com/msp/signcerts/cert.pem \
organizations/peerOrganizations/org1.example.com/\
users/AuditorUser@org1.example.com/msp/admincerts/
```

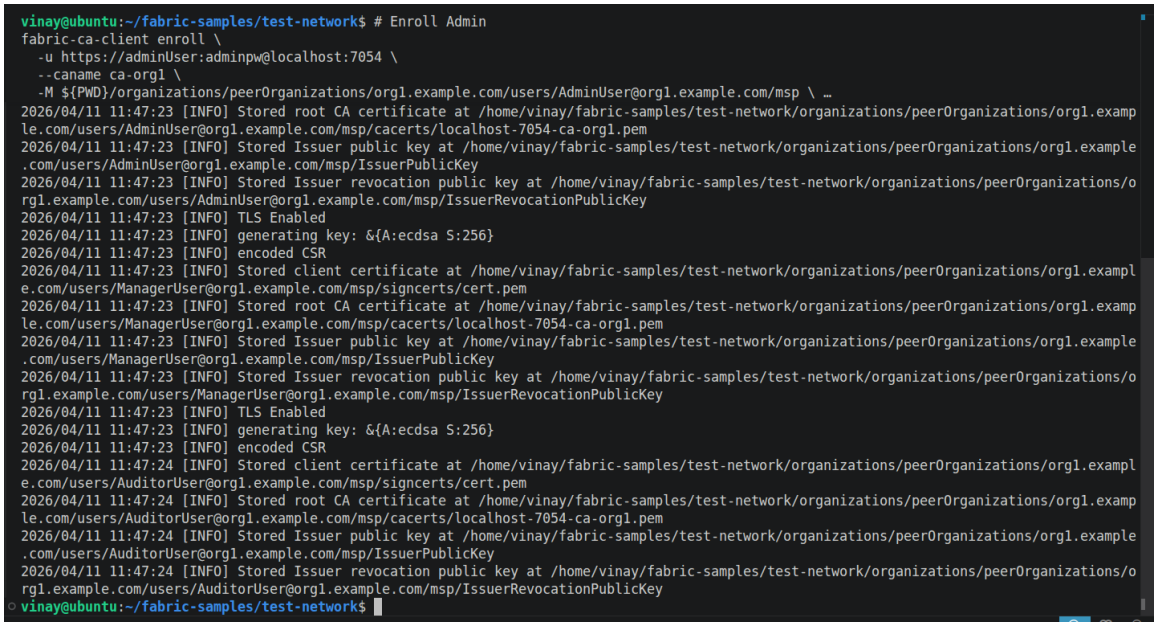


Figure 7

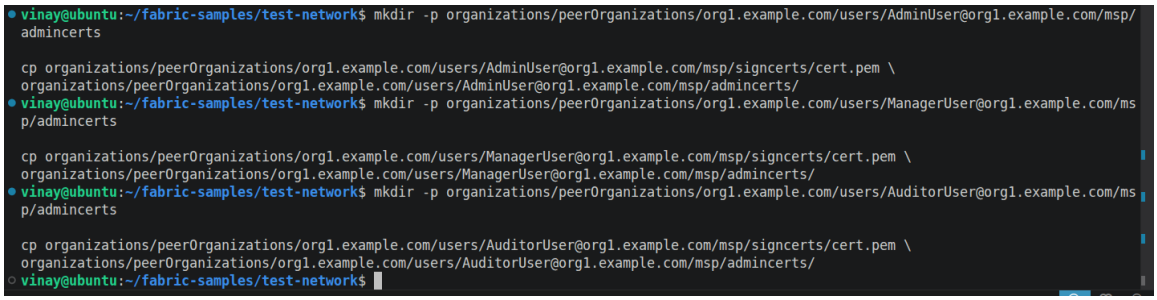


Figure 8

8 Testing and Results

Before every test, switch the active identity by exporting:

`export CORE_PEER_MSPCONFIGPATH=.../users/{RoleUser}@org1.example.com/msp`

8.1 Test 1 — Admin Creates an Asset

Listing 14: Admin: CreateAsset

```
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/
  peerOrganizations/\
 org1.example.com/users/AdminUser@org1.example.com/msp
```

```
peer chaincode invoke \
  -o localhost:7050 --ordererTLSHostnameOverride orderer.
     example.com \
  --tls --cafile "$ORDERER_CA" \
  -C mychannel -n rbac \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/\
    org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/\
    org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"CreateAsset","Args":["ASSET001","Laptop
     ","50000"]}'

sleep 3
```

Expected result: `chaincode response: 200` — Asset ASSET001 created successfully.

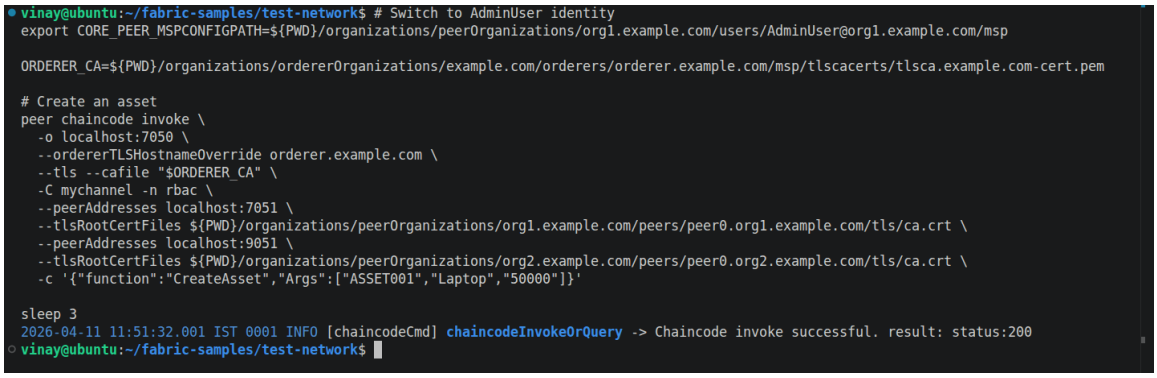


Figure 9

8.2 Test 2 — Auditor Reads an Asset

Listing 15: Auditor: ReadAsset

```
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/
   peerOrganizations/\
  org1.example.com/users/AuditorUser@org1.example.com/msp

peer chaincode query \
  -C mychannel -n rbac \
  -c '{"function":"ReadAsset","Args":["ASSET001"]}'
```

Expected result: `{"id":"ASSET001","name":"Laptop","value":"50000","owner":"Org1MSP"}`

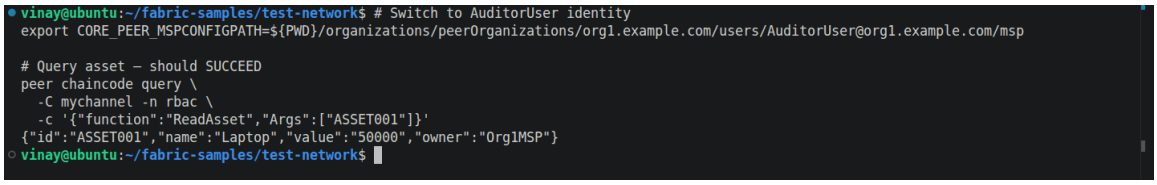


Figure 10

8.3 Test 3 — Auditor Tries to Create (Unauthorized)

Listing 16: Auditor attempts CreateAsset — should be rejected

```
# Auditor identity still active

peer chaincode invoke \
  -o localhost:7050 --ordererTLSHostnameOverride orderer.
     example.com \
  --tls --cafile "$ORDERER_CA" \
  -C mychannel -n rbac \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/\
    org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/\
    org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"CreateAsset","Args":["ASSET002","Phone
     ","20000"]}'
```

Expected error:

```
Error: endorsement failure during invoke. response: status:500
message:"Access denied: only Admin can create assets (your role
   : Auditor)"
```

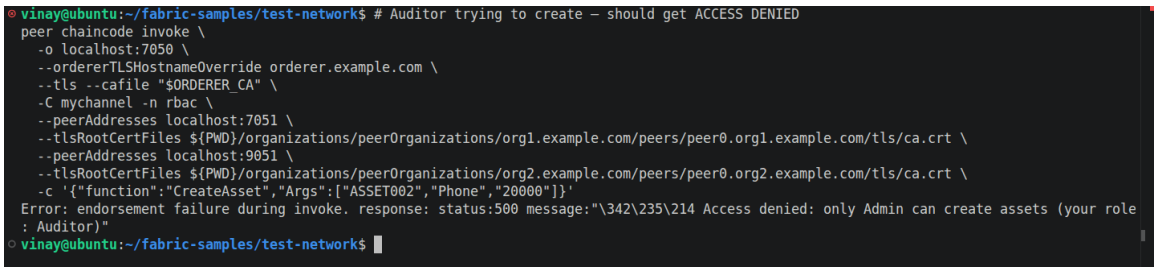


Figure 11

8.4 Test 4 — Manager Updates an Asset

Listing 17: Manager: UpdateAsset

```
export CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/
   peerOrganizations/\
  org1.example.com/users/ManagerUser@org1.example.com/msp

peer chaincode invoke \
  -o localhost:7050 --ordererTLSHostnameOverride orderer.
     example.com \
  --tls --cafile "$ORDERER_CA" \
  -C mychannel -n rbac \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/\
    org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles ${PWD}/organizations/peerOrganizations/\
    org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  -c '{"function":"UpdateAsset","Args":["ASSET001","Laptop Pro
     ","75000"]}'

sleep 3

# Verify the update
peer chaincode query -C mychannel -n rbac \
  -c '{"function":"ReadAsset","Args":["ASSET001"]}'
```

Expected result: Asset updated. Query returns `"name":"Laptop Pro","value":"75000"`.

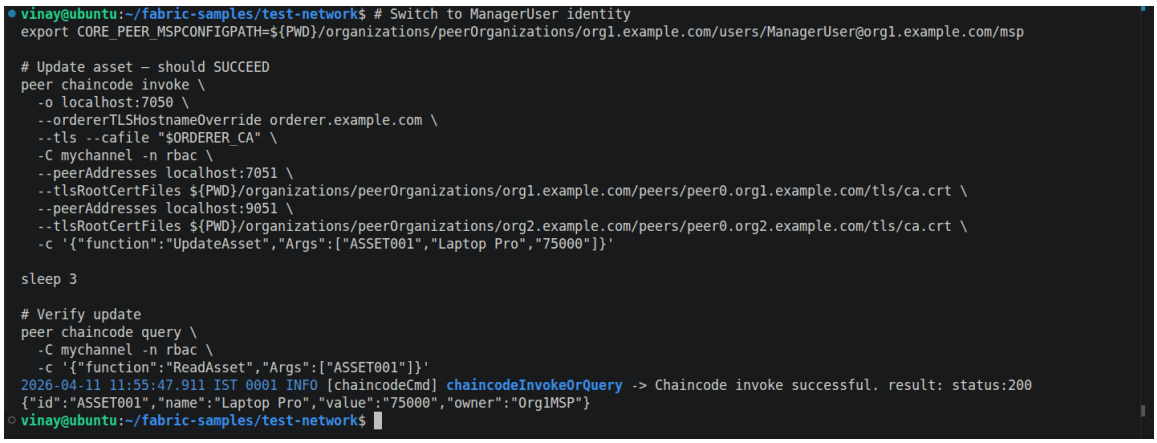


Figure 12

8.5 Test 6 — Frontend Demo

Listing 18: Start backend and open frontend

```
# Terminal 1 --- start backend
cd ~/rbac-project/backend
npm install
node server.js

# Terminal 2 --- open UI
python3 -m http.server 8000
```

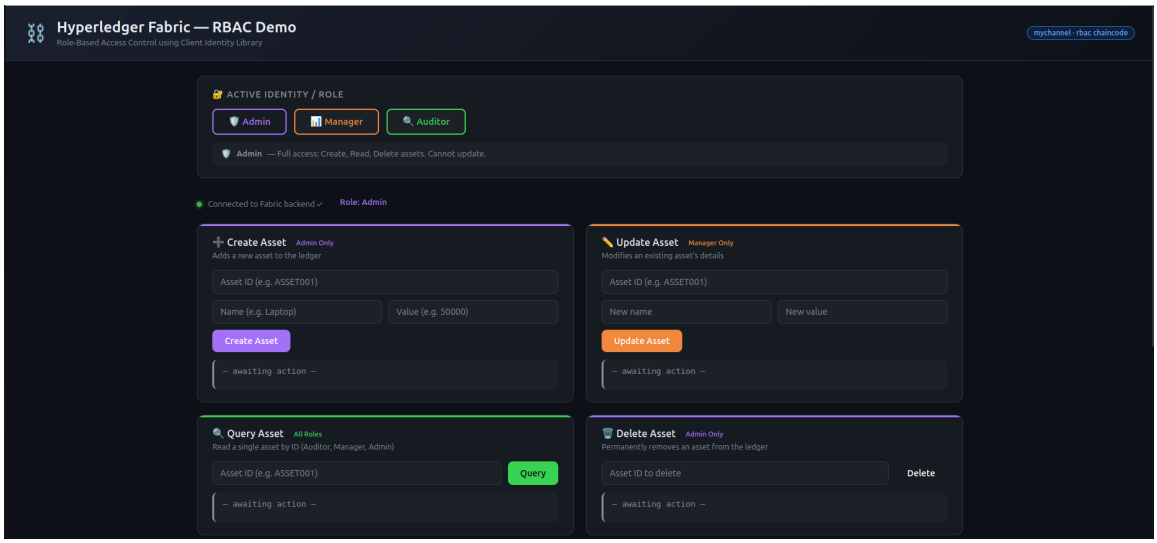


Figure 13

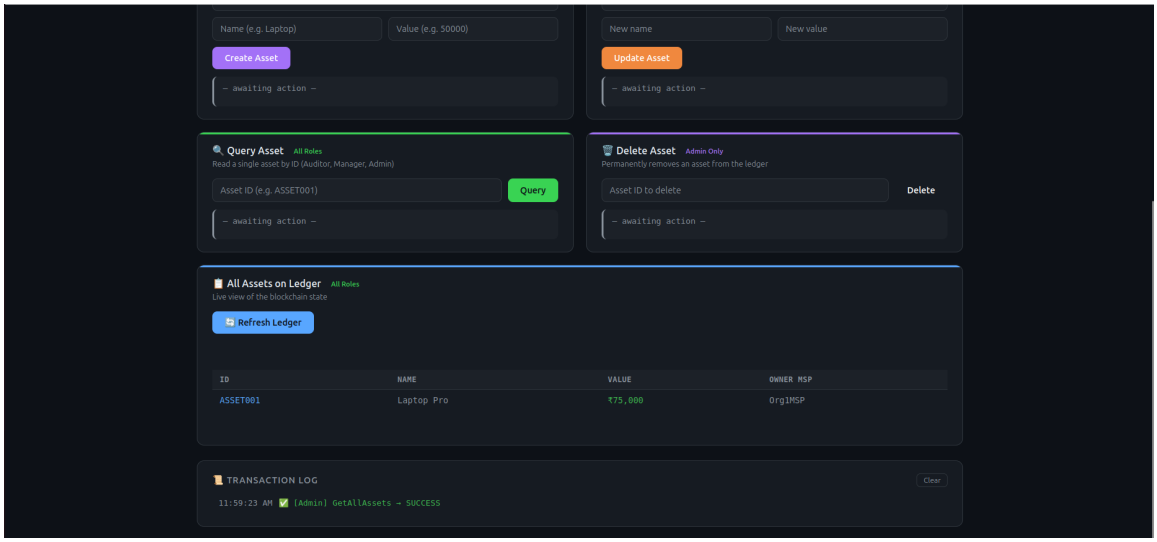


Figure 14

9 Conclusion

This assignment successfully demonstrated the implementation of Role-Based Access Control in Hyperledger Fabric using the Client Identity Library. The key outcomes are:

1. **Admin** can create and delete assets. All other write operations are denied.
2. **Manager** can update existing assets but cannot create or delete.
3. **Auditor** has read-only access. All write operations are rejected with a clear error.
4. **Unauthorized operations** are rejected at the chaincode level, making the system secure even against direct CLI attacks.
5. The **role attribute** is cryptographically bound to the user's X.509 certificate via the Fabric CA, making it tamper-proof.

This approach is production-ready: roles are enforced at the smart contract level, the audit trail is immutable on the blockchain, and the system integrates cleanly with the standard Fabric CA enrollment workflow.

GitHub Repository:

`https://github.com/vinay-0821/HyperLedger-RBAC`